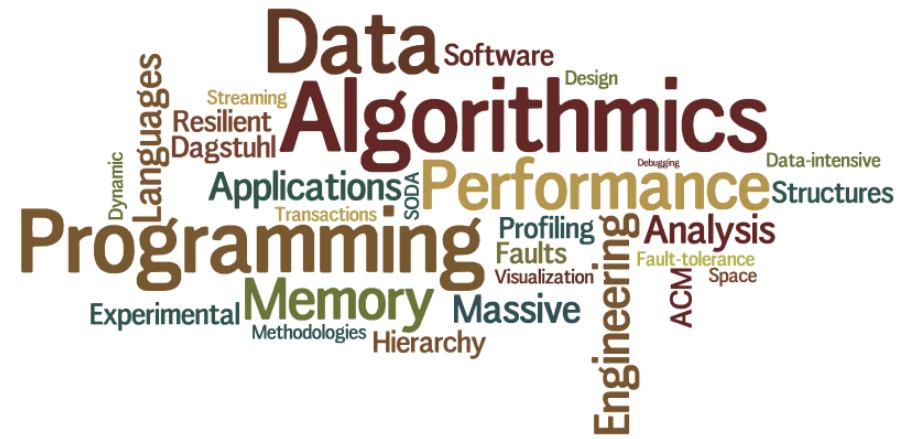


Design and Analysis of Algorithm: (BCSE204L)

Introduction to Algorithms



Evelin Nissy Thomas
Assistant Professor (Junior)
SCOPE, VIT Vellore

Text Books:

1. Thomas H. Cormen, C.E. Leiserson, R L. Rivest and C. Stein, Introduction to Algorithms, 2009, 3rd Edition, MIT Press.

Reference Books:

1. Jon Kleinberg, Éva Tardos ,Algorithm Design, Pearson education, 2014.
2. Rajeev Motwani, Prabhakar Raghavan; Randomized Algorithms, Cambridge University Press, 1995 (Online Print – 2013)
3. Ravindra K.Ahuja, Thomas L.Magnanti and James B. Orlin, “ Network Flows: Theory, Algorithms and Applications”, Pearson Education, 2014.

Module I: Design Paradigms: Greedy, Divide and Conquer Techniques(6 Hours)

- **Overview and Importance of Algorithms**
 - **Stages of algorithm development:**
 - Describing the problem
 - Identifying a suitable technique
 - Design of an algorithm
 - Derive Time Complexity,
 - Proof of Correctness of the algorithm
- **Illustration of Design Stages**
 - **Greedy techniques:**
 - Fractional Knapsack Problem
 - Huffman coding
 - **Divide and Conquer:**
 - Maximum Subarray
 - Karatsuba faster integer multiplication algorithm.

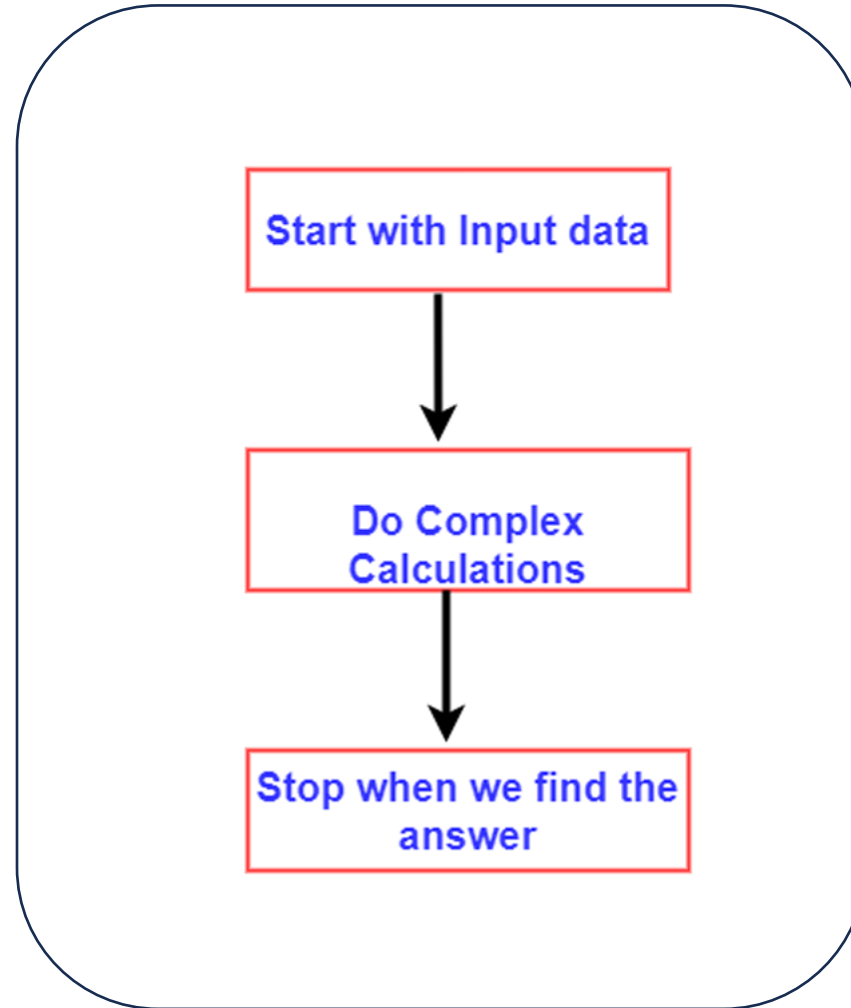
What is Algorithm?

Algorithm: A set of steps to accomplish a particular task.

Examples:

- A step-by-step algorithm for preparing a dish. Input = ingredients, Output = finished meal.
- Google Maps directions: Uses shortest path algorithms to find the quickest route.
- To choose a gadgets (Mobile/PC/ Tablets) to buy.
- Netflix recommendations
- Social media feed

Algorithms



- **Input-** There should be 0 or more inputs supplied externally to the algorithm.
- **Output-** There should be at least 1 output obtained.
- **Definiteness-** Every step of the algorithm should be clear and well defined.
- **Finiteness-** Algorithm should have finite number of steps.
- **Correctness-** Every step of the algorithm must generate a correct output.
- **An algorithm is said to be efficient and fast, if it takes less time to execute and consumes less memory space.**
 - Time Complexity
 - Space Complexity

Stages of algorithm development

Stages of algorithm development for solving a problem:

- Describing the problem.
- Identifying a suitable technique.
- Design of an algorithm.
- Proof of Correctness of the algorithm.

Describing the problem & Identifying methods

Example:- Finding the average of list of n numbers.

- **Input:-** The list of n numbers
- **Output:-** The average
- **Method:-**
 - Sum all elements and store in sum
 - Divide the sum by n which is the average. Store it in avg
 - Output the Average avg

Design of an algorithm.

Can be done in English, pseudocode, flowchart so on..

1. Receive n as input
2. Receive n numbers as input
3. Find the sum of n numbers and store in a variable sum
4. Compute $avg = \frac{sum}{n}$
5. Output avg

Proof of Correctness of the algorithm.

- Correct:- if, for every input instance, it halts with the correct output
- An incorrect algorithm
 - may not halt at all for some input instances, or
 - may halt with a wrong output
- Correct if the algorithm yields the desired output for the specific input in a finite amount of time.
- Proof of correctness depends on the algorithm

Proof of Correctness of the algorithm.

- For some algorithms, the proof of correctness is easy and for some algorithms, it is complex
- It depends on the type of instructions in the algorithm
- In order to show that an algorithm is incorrect, one instance of input is sufficient for which the algorithm fails to give the desired output

Proof of Correctness of the algorithm

- Sorting Problem: Insertion Sort

Input: A sequence of n numbers $\langle a_1, a_2, \dots, a_n \rangle$.

Output: A permutation (reordering) $\langle a'_1, a'_2, \dots, a'_n \rangle$ of the input sequence such that $a'_1 \leq a'_2 \leq \dots \leq a'_n$.



- Insertion sort works in the way many people sort a hand of playing cards.

- Insertion Sort Algorithm

Current element which is to be inserted

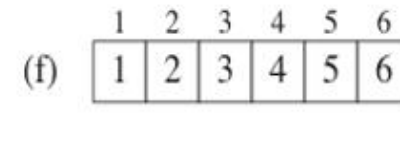
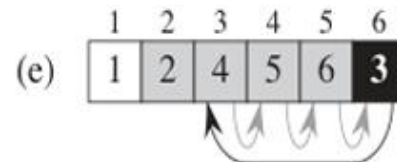
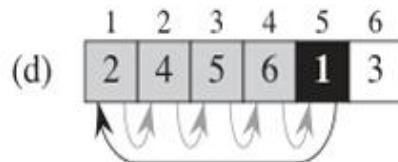
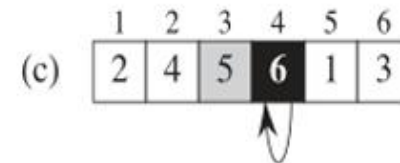
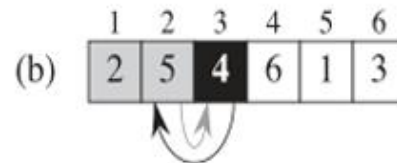
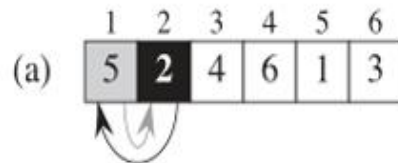
INSERTION-SORT(A)

```

1  for  $j \leftarrow 2$  to  $A.length$ 
2       $key \leftarrow A[j]$ 
3      // Insert  $A[j]$  into the sorted sequence  $A[1..j-1]$ .
4       $i \leftarrow j - 1$ 
5      while  $i > 0$  and  $A[i] > key$ 
6           $A[i+1] \leftarrow A[i]$ 
7           $i \leftarrow i - 1$ 
8       $A[i+1] \leftarrow key$ 
    
```

Sorted Subsequence

$A[j+1..n]$ - remaining numbers



Proof of Correctness of the algorithm

Loop Invariant Method:

- A loop invariant is a condition (among program variables) that is necessarily true immediately before and immediately after each iteration of a loop.
- In other words, it is some condition that holds for every iteration of the loop.

Example:

Consider an Array $A = \{8, 6, 4, 11, 3, 7\}$ with 6 elements and we have to find maximum element “max” in the array.

Proof of Correctness of the algorithm

- Consider an Array $a=\{8,6,4,11,3,7\}$ with 6 elements and we have to find maximum element “max” in the array.

Alg max(a[],max)

```
{  
    for(i=0;i<6;i++)  
    {  
        if(a[i]>max)  
            max=a[i]  
    }  
}
```

Here, loop invariant condition is that “Max” is always maximum among the first “i” elements of the array A



Proof of Correctness of the algorithm

In **loop invariant method** of proving correctness, one must show three things about a loop invariant.

- **Initialization:** It is true prior to the first iteration of the loop.
- **Maintenance:** If it is true before an iteration of the loop, it remains true before the next iteration.
- **Termination:** When the loop terminates, the invariant gives us a useful property that helps show that the algorithm is correct.

Proof of Correctness of the algorithm

Initialization:

- To show that the loop invariant holds before the first loop iteration
- When $j = 2$, the subarray $A[1..j - 1]$ consists of just the single element $A[1]$
- Which is the original element in $A[1]$
- This subarray is trivially sorted.

Shows that Loop invariant holds prior to the first iteration of the loop.

Proof of Correctness of the algorithm

Maintenance : Each iteration maintains the loop invariant.

- The body of the for loop works by moving $A[j - 1]$, $A[j - 2]$, $A[j - 3]$, and so on by one position to the right until it finds the proper position for $A[j]$ (lines 4–7), at which point it inserts the value of $A[j]$ (line 8).
- The subarray $A[1..j]$ then consists of the elements originally in $A[1..j]$, but in sorted order.
- Incrementing j for the next iteration of the for loop then preserves the loop invariant

Proof of Correctness of the algorithm

Termination : What happens when the loop terminates??

- The condition causing the for loop to terminate is that $j > A.length = n$
- Substituting $n+1$ for j in the wording of loop invariant, we have that the subarray $A[1..n]$ consists of the elements originally in $A[1..n]$, but in sorted order
- As $A[1..n]$ is the original array it implies that the entire array is sorted

Hence the algorithm is correct.